

Rezumat Curs 5 — Reprezentări Numerice ale Datelor în ML

Surse: 05_Reprezentari_numerice_ale_datelor.pdf + 5_2_Reprezentari_numerice_ale_datelor_in_ML.md UBB Informatică Anul 2 — Inteligență Artificială

1. Principiul de bază

Modelele de ML lucrează exclusiv cu numere. Orice tip de date — text, imagini, audio — trebuie transformat într-o reprezentare numerică înainte de a fi procesat.

Un model ML este o **funcție matematică** de forma:

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^k$$

Adică primește n numere ca intrare și returnează k numere ca ieșire, care sunt ulterior interpretate (ex: pixel, clasă, probabilitate).

2. Funcții matematice în ML — Tipuri

Tip	Notăție	Exemplu
1 intrare, 1 ieșire	$f : \mathbb{R} \rightarrow \mathbb{R}$	$f(x) = x^2 + 2x + 1, f(3) = 16$
2 intrări, 1 ieșire	$f : \mathbb{R}^2 \rightarrow \mathbb{R}$	$f(x, y) = x^2 + y^2 + xy, f(2, 3) = 19$
1 intrare, 2 ieșiri	$f : \mathbb{R} \rightarrow \mathbb{R}^2$	$f(x) = (x^3 + 3, x - 4), f(2) = (11, -2)$
n intrări, k ieșiri	$f : \mathbb{R}^n \rightarrow \mathbb{R}^k$	Model RNA general

3. Modelul RNA din perspectiva funcțiilor

O rețea neuronală este o funcție matematică $f_{\text{arc}} : \mathbb{R}^{n+p+q} \rightarrow \mathbb{R}^m$ cu:

- p **weights** (ponderi): $w = [w_1, \dots, w_p]$
- q **biases**: $b = [b_1, \dots, b_q]$
- O funcție de **cost (loss)**: $\text{loss} : \mathbb{R}^m \times \mathbb{R}^m \rightarrow \mathbb{R}$

Pseudocod:

```
predict(x):  
    return f_arc(x, w, b)  
  
train(x, y):  
    pred = f_arc(x, w, b)  
    L = loss(pred, y)
```

calculezi $\partial L / \partial w_i$ și $\partial L / \partial b_j$
actualizezi w, b cu gradient descent

4. Reprezentarea Imaginilor

4.1 Structura unei imagini

O imagine = o matrice de pixeli:

- **Alb-negru:** 1 valoare de intensitate per pixel, interval $[0, 255]$
- **Color (RGB):** 3 valori per pixel (Red, Green, Blue), fiecare în $[0, 255]$

Exemplu: Cod HEX **#1B5E20** → Roșu: 27, Verde: 94, Albastru: 32

4.2 Calculul dimensiunii vectorului

$$\text{nr. valori} = \text{lățime} \times \text{înălțime} \times 3$$

Exemple:

Imagine	Pixeli	Valori totale
13×13	169	$169 \times 3 = 507$
1024×1024	1.048.576	3.145.728

Deci pentru DALL-E (generează imagini 1024×1024):

$$f(\dots) = (y_1, y_2, \dots, y_{3.145.728})$$

5. Reprezentarea Textului

5.1 Problema indexării simple

Dacă atribuim fiecărui cuvânt un index (câine=1, pisică=2, fereastră=3), modelul ar crede că „câine” și „pisică” sunt mai similare cu „fereastră” decât între ele — ceea ce e greșit. Indexarea simplă **nu păstrează semantica**.

5.2 Soluția: Embedding-uri

Embedding = un vector dens de numere reale asociat unui cuvânt, învățat prin ML:

$$\text{Emb}(\text{"word"}) = (x_1, x_2, \dots, x_k)$$

Proprietăți ale embedding-urilor de calitate:

- Păstrează distanța semantică: $\text{dist}(\text{Emb}(\text{câine}), \text{Emb}(\text{pisică})) < \text{dist}(\text{Emb}(\text{câine}), \text{Emb}(\text{fereastră}))$
- Ortogonalitate pentru cuvinte neconectate: $\text{Emb}(\text{ușă}) \perp \text{Emb}(\text{pește})$
- Independente de lungimea propoziției → putere de generalizare

Exemplul clasic: $\text{Emb}(\text{king}) - \text{Emb}(\text{queen}) \approx \text{Emb}(\text{man}) - \text{Emb}(\text{woman})$

5.3 Tokenizare

Deoarece nu putem stoca un embedding pentru fiecare propoziție posibilă, textul se împarte în **tokeni** (cuvinte sau subcuvinte):

1. $\text{Tokenize}(\text{text}) \rightarrow (t_1, t_2, \dots, t_q)$
2. $\text{map}(\text{Emb}, \text{tokens}) \rightarrow (x_{1,1..k}, x_{2,1..k}, \dots, x_{q,1..k})$

6. Tipuri de Embedding-uri

6.1 Frequency-based embeddings

Bazate pe frecvența de apariție a cuvintelor în texte. Simple, rapide.

6.2 Prediction-based embeddings (Word2Vec, CBOW)

Învățate de o RNA pentru prezicerea cuvântului următor sau a cuvântului din context.

Arhitectura CBOW (Continuous Bag of Words):

- Intrare: vectori one-hot ai cuvintelor de context
- Strat ascuns: stratul de embedding (N -dimensional)
- Ieșire: softmax pentru predicția cuvântului țintă

Exemplu: Din contextul [she, ride] → prezice "bike"

$$\hat{v} = \frac{v_{\text{she}} + v_{\text{ride}}}{2} \rightarrow W'^T \cdot \hat{v} = z \rightarrow \hat{y} = \text{softmax}(z)$$

6.3 Positional embeddings

Indică **poziția unui token în propoziție** — esențiale pentru Transformere care nu procesează secvențial.

Tip	Problemă	Distanță
Poziție efectivă (1,2,3...)	Schimbă cu lungimea propoziției	Monotonă ✓
Normalizată (pos/N)	Al k-lea cuvânt diferit în propoziții diferite	Monotonă ✓
One-hot encoding	$\text{sim}(w_i, w_j) = 1.4$ pentru orice $i \neq j$ — toate la fel de diferite!	Fixă ✕
Binary encoding	Distanța nu e monotonă	Neuniformă ✕
Sin-based (RoPE)	Soluția preferată în Transformere	Continuă ✓

Sin-based encoding (formula)

$$PE(\text{pos}, i) = \sin\left(\frac{2\pi \cdot \text{pos}}{i}\right), \quad i = 0, 1, \dots, d-1$$

Unde:

- pos = poziția tokenului în propoziție
- i = dimensiunea embedding-ului (frecvența)
- d = dimensiunea totală a embedding-ului

7. Modelul Complet pentru Generare Imagini din Text

Exemplu complet (DALL-E stil):

input: text

```
(z1, z2, ..., zp) = generateRandomNumbers()    ← vectorul latent (zgomot)
(t1, t2, ..., tq) = Tokenize(text)             ← tokenizare
(x1,1..k, ..., xq,1..k) = map(Emb, tokens)       ← embedding tokeni
(y1, ..., yk) = f(x1..xq, z1..zp)          ← modelul principal
image = imageFromVector(y1, ..., yk)          ← reconstrucție imagine
```

output: image

Unde:

- z = vectorul latent — adăugând zgomot aleator, obținem rezultate diferite la fiecare rulare (generativitate)
- f = funcția dificilă — **modelul antrenat** (aceasta e partea grea)
- `generateRandomNumbers()`, `Tokenize()`, `Emb()`, `imageFromVector()` = componente deja implementate

Rolul spațiului latent: Variații mici în $z \rightarrow$ variații mici în imaginea generată. Controlează culori, poziție, stil.

8. Concepte-cheie de memorat

1. **ML lucrează doar cu numere** $\rightarrow$ orice date trebuie transformate numeric
2. **Imaginea** = matrice de pixeli $\rightarrow$ lățime $\times$ înălțime $\times$ 3 valori RGB în $[0, 255]$
3. **Indexare simplă** = nu păstrează semantica
4. **Embedding** = vector dens învățat care păstrează relații semantice
5. **Tokenizare** = împărțirea textului în tokeni pentru a putea atribui embeddings
6. **One-hot encoding** = toate cuvintele la aceeași distanță unele de altele $\rightarrow$ problematic pentru poziții
7. **Sin-based encoding** = soluția preferată pentru positional embeddings în Transformere
8. **Spațiu latent** = vectorul aleator z care controlează variabilitatea output-ului generativ

9. ⚠️ Aspecte Critice pentru Examen

Subiect 18 — "Care afirmație este adevărată pentru embedding-urile obținute prin one-hot encoding?"

Răspuns corect: **a. Vectorii rezultanți conțin valori de 0 și 1** — one-hot produce exact asta.

Răspunsul greșit de evitat: **b** (nu surprind relația semantică) și **c** (nu sunt ineficiente neapărat pentru vocabulare scurte).

Concepte frecvente la întrebări de tip grilă:

- One-hot encoding → valori 0 și 1, nu păstrează semantica
- Embedding-uri → valori reale, păstrează relații semantice
- RGB → 3 valori per pixel, $[0, 255]$
- Formula dimensiunii imaginii: $W \times H \times 3$

Rezumat generat exclusiv din *05_Reprezentari_numerice_ale_datelor.pdf* și *5_2_Reprezentari_numerice_ale_datelor_in_ML.md*